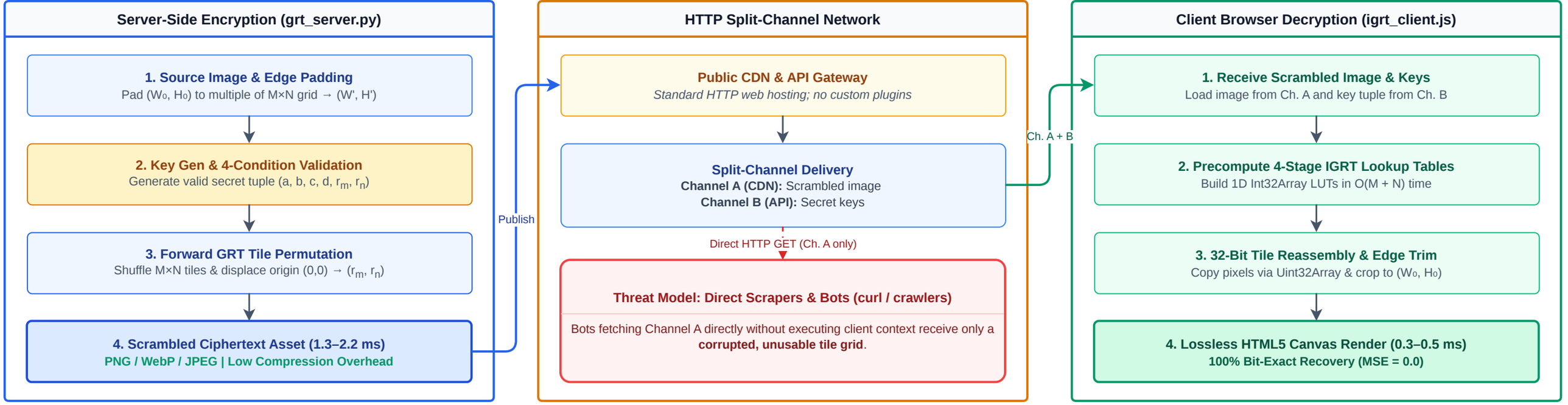

Server-Side Encryption (grt_server.py)
1. Source Image & Edge Padding
Pad (W₀, H₀) to multiple of M×N grid → (W', H')
2. Key Gen & 4-Condition Validation
Generate valid secret tuple (a, b, c, d, r_m, r_n)
3. Forward GRT Tile Permutation
Shuffle M×N tiles & displace origin (0,0) → (r_m, r_n)
4. Scrambled Ciphertext Asset (1.3–2.2 ms)
PNG / WebP / JPEG | Low Compression Overhead
Publish
HTTP Split-Channel Network
Public CDN & API Gateway
Standard HTTP web hosting; no custom plugins
Split-Channel Delivery
Channel A (CDN): Scrambled image
Channel B (API): Secret keys
Direct HTTP GET (Ch. A only)
Threat Model: Direct Scrapers & Bots (curl / crawlers)
Bots fetching Channel A directly without executing client context receive only a corrupted, unusable tile grid.
Ch. A + B
Client Browser Decryption (igrt_client.js)
1. Receive Scrambled Image & Keys
Load image from Ch. A and key tuple from Ch. B
2. Precompute 4-Stage IGRT Lookup Tables
Build 1D Int32Array LUTs in O(M + N) time
3. 32-Bit Tile Reassembly & Edge Trim
Copy pixels via Uint32Array & crop to (W₀, H₀)
4. Lossless HTML5 Canvas Render (0.3–0.5 ms)
100% Bit-Exact Recovery (MSE = 0.0)